```
!pip install nltk scikit-learn -q
import nltk
nltk.download('movie_reviews', quiet=True)
import random
from nltk.corpus import movie_reviews
from sklearn.model_selection import train_test_split
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.naive_bayes import MultinomialNB
from sklearn.metrics import (accuracy_score, precision_score, recall_score,
                             f1_score, classification_report, confusion_matrix)
```

```
# ------------------------------------------------------------
# Load the built-in NLTK movie_reviews corpus
# (2000 reviews total: 1000 positive, 1000 negative)
# ------------------------------------------------------------
documents = [
    (" ".join(movie_reviews.words(fileid)), category)
    for category in movie_reviews.categories()
    for fileid in movie_reviews.fileids(category)
]
random.seed(42)
random.shuffle(documents)
texts = [text for text, label in documents]
labels = [label for text, label in documents]
print(f"Total reviews: {len(texts)}")
print(f"Positive: {labels.count('pos')}, Negative: {labels.count('neg')}")
```

```
Total reviews: 2000
Positive: 1000, Negative: 1000
```

```
# ------------------------------------------------------------
# a. Bag-of-Words feature extraction
# ------------------------------------------------------------
X_train_text, X_test_text, y_train, y_test = train_test_split(
    texts, labels, test_size=0.2, random_state=42, stratify=labels
)
vectorizer = CountVectorizer(stop_words='english', max_features=5000)
X_train = vectorizer.fit_transform(X_train_text)
X_test = vectorizer.transform(X_test_text)
print(f"\nVocabulary size (BoW features): {len(vectorizer.vocabulary_)}")
print(f"Training matrix shape: {X_train.shape}")
```

```
Vocabulary size (BoW features): 5000
Training matrix shape: (1600, 5000)
```

```
# ------------------------------------------------------------
# b. Train a Naive Bayes classifier
# ------------------------------------------------------------
clf = MultinomialNB()
clf.fit(X_train, y_train)
```

```
▾ MultinomialNB
MultinomialNB()
```

```
# ------------------------------------------------------------
# c. Evaluate the model
# ------------------------------------------------------------
y_pred = clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
precision = precision_score(y_test, y_pred, pos_label='pos')
recall = recall_score(y_test, y_pred, pos_label='pos')
```

```
f1 = f1_score(y_test, y_pred, pos_label='pos')
print("\n=== Model Performance ===")
print(f"Accuracy  : {accuracy:.4f}")
print(f"Precision : {precision:.4f}")
print(f"Recall    : {recall:.4f}")
print(f"F1 Score  : {f1:.4f}")
print("\n=== Detailed Classification Report ===")
print(classification_report(y_test, y_pred, target_names=['neg', 'pos']))
print("\n=== Confusion Matrix ===")
print(confusion_matrix(y_test, y_pred, labels=['neg', 'pos']))
```

```
=== Model Performance ===
Accuracy  : 0.7850
Precision : 0.8098
Recall    : 0.7450
F1 Score  : 0.7760

=== Detailed Classification Report ===
              precision    recall  f1-score   support

         neg       0.76      0.82      0.79       200
         pos       0.81      0.74      0.78       200

    accuracy                           0.79       400
   macro avg       0.79      0.78      0.78       400
weighted avg       0.79      0.79      0.78       400


=== Confusion Matrix ===
[[165  35]
 [ 51 149]]
```

```
# ----------------------------------------------------------
# Try the classifier on a couple of custom sentences
# ----------------------------------------------------------
sample_reviews = [
    "This movie was absolutely fantastic, brilliant acting and a gripping story!",
    "Terrible film, a complete waste of time. Boring plot and bad acting."
]
sample_features = vectorizer.transform(sample_reviews)
sample_preds = clf.predict(sample_features)
print("\n=== Predictions on Custom Reviews ===")
for review, pred in zip(sample_reviews, sample_preds):
    print(f"Review: {review}\nPredicted Sentiment: {pred}\n")
```

```
=== Predictions on Custom Reviews ===
Review: This movie was absolutely fantastic, brilliant acting and a gripping story!
Predicted Sentiment: pos

Review: Terrible film, a complete waste of time. Boring plot and bad acting.
Predicted Sentiment: neg
```